Name: Jaykishan J Saharan
Div:- D Roll no.:- 26
course/Program:- BTech CSE SY
Subject :- DSA

PSA lab assingment 4.

* Title: write a c++ program to sort an array of N elements using Quick Sort.

* Aim: write a c++ program to sort an array of N elements using Quick Sort. Also write how to determine time complexity and Space complexity of your program.

Q.1]. What is quicksort algorithm? Explain how quick sort works with an example array: [8,4,7,9,3,10,5].

Sol:- Quick Sort is a divide-and-conquer sorting algorithm

→ It works by selecting a pivot element from the array then partitioning the array into two subarrays.

→ It works by by selecting a pivot element from the array, then partitioning the array into two subarrays:

- Elements smaller than pivot
- Elements greater than pivot
- The process is recursively applied to the subarrays until the array is sorted.

Array: [8,4,7,9,3,10,5]

steps: 1) choose a pivot (say last element = 5).

Step: 2) Partition array

- Elements smaller than 5: [4,3]
  Pivot = 15
- Elements greater than 5: [8,7,9,10]
  → after partition [4,3,5,8,7,9,10]

Step: 3) Recursively apply quicksort:

- left [4,3] → becomes [3,4]
- right [8,7,9,10]
- Pivot = 10, Partition → [8,7,9,10]
- Quick Sort [8,7,9] → [7,8,9]

Step: 4) final Quick sorted array = [3,4,5,7,8,9,10]

q.3) Applications of Quick Sort?

Sol^n:- General purpose Sorting:-(most programming language use it internally with optimizations, eg., Python's Timesort has Quick logic).

- Databases: Sorting large databases efficiently.
- Search algorithms: Helps in divide and Conquer based Searching.
- Information retrivel: used in indexing / search engine
- Graphics: Sorting objects by depths (z-buffering).

q.4) Pseudocode for Quick Sort?

Sol^n:-

```
Quicksort (A, low, high):
  if low < high:
  Pivotindex = Partition (A, low, high)
  Quicksort (A, low, Pivotindex - 1)
  QuickSort (A, Pivotindex + 1, high)
  Partition (A, low, high):
  pivot = A[High]
  i = low - 1
```

```
for j = low to high - 1 :
if A[j] <= pivot :
  i = i + 1
  swap A[i] and A[j]
swap A[i+1] and A[high]
 return (i+1)
```

Q.5) Given the array [29,10,14,37,13] perform one full pass of Quicksort using the last element as pivot. Show the resulting array after this pass.

Sol^n :- Array: [29,10,14,37,13]

- Pivot = 13
- Partitioning :

compare each with 13 :-

- $29 > 13$ → stays
- $10 < 19$ → moves before pivot
- $14 > 13$ → stays
- $37 \geq 13$ → stays

After swapping pivot into correct place :

→ [10,13,14,29,37]

Q.6) compare the best-worst and worst-case time complexity of Quick sort. Under what conditions do these cases occurs.

Sol^n :- Best case : $O(n \log n)$

- occurs when pivot splits array into nearly equal halves at every step.

Worst case : $O(n^2)$

- occurs when pivot is always the smallest / largest elements (very unbalanced partition).

Avg case : $O(n \log n)$

Q.7) Give two sorting algorithms-Quick Sort and Merge Sort. Discuss which algorithm would be more efficient for sorting a large dataset that doesn't fit into memory and justify your answer.

Sol^ :- Quick Sort :

- In place (needs only $o(\log n)$ extra stack space)
- Very cache friendly.
- But not-stable and worst case is $O(n^2)$.

Merge Sort :

- Requires extra memory $o(n)$.
- Stable Sort.
- Very suitable for external sorting (data on display since it can merge sorted chuncks efficiently.).

* Conclusion :

Thus, we have implemented Quick Sort using array in C++.

15/10/25